SAMS Kernel

The programmer's guide: writing TI-99/4A applications in assembly

Version 1.5.2 · 28 October 2026
For a TI-99/4A with an F18A video card and a SAMS memory card
MIT License, © 2026 Rebecca Sutter

Contents

0. What you get, what you need

1. Hello, kernel

2. The machine, in one page

3. The anatomy of a module

4. Calling conventions and registers

5. The screen

6. The keyboard

7. Memory

8. Several modules

9. Files

10. Time

11. Quitting

12. Building

13. Testing

14. The checklist: what has already cost a day

15. What the kernel does not do (version 1.5)

Appendix: a second module in five minutes

How to write an application for the TI-99/4A in TMS9900 assembly on the SAMS Kernel. The guide is a tutorial first (parts 1-3 get a program on the screen), then a tour of each service (parts 4-11), then the practical matter of building and testing (parts 12-13), and ends with a checklist of the mistakes that have already cost someone a day (part 14). Look up a single routine in ROUTINES.md; the machine's facts are in HARDWARE.md; how the kernel works inside is in ARCHITECTURE.md.

The kernel's source is the one place that is never out of date. Where this guide and a header comment in `kernel/` disagree, the comment is right and this guide should be fixed.

0. What you get, what you need

The machine the kernel assumes: a TI-99/4A with an **F18A** (or PICO9918) video card, a **SAMS** memory card of at least the size your application asks for (128 pages, 512K, by default), and a cartridge slot holding a **FinalGROM 99** (or any cartridge that presents 8K banks selected by writes to `>6000 + 2n`: the "8" image format; js99er and Classic99 load the same file). The loader refuses to start, with a plain message, if the card, its size or the F18A is missing.

What the kernel gives your program:

Screen	boots in 80 x 30 text (F18A "Text 80", a colour per character cell; 256 six-by-eight glyphs; 15 colour *roles* and five colour schemes; DMA row moves); `VMODE` sets **any F18A mode**: Text 80/40, Graphics I and II, Multicolor, enhanced colour, sprites, tile layer 2, the bitmap layer (part 5 and VIDEO-MODES.md)
Keyboard	64-key buffer, auto-repeat, SHIFT / CTRL / FCTN / ALPHA LOCK chords decoded to one byte each
Memory	7K of RAM that is always there (slot A), 8K of your code in the window at `>C000`, and up to 16M of SAMS pages you map into `>E000-FFFF`; a page pool to hand them out
Structure	a main loop that calls your module once a frame; up to dozens of 8K modules; `GOMOD` to switch, `MODCALL` to call into another module and come back
Files	open, read, write, close, delete through any DSR (disk controller, TIPI, ...)
Music	play a song from SAMS pages (a register-log player, stepped every frame), tone effects; songs compile from MIDI or a text score (MUSIC.md)
Safety	the loader's checks, a clean power-up state, a clean exit to the TI title screen

What you need on the PC (the **[PC]** route): Python 3, and xdt99 (`xas99.py` is the assembler). Put xdt99 beside this directory (`../xdt99`), or set `XAS99` to the path of `xas99.py`. A web browser with js99er.net for running; the headless simulator in `tools/` for testing.

What you need on the TI

[TI] The workbench cartridge image (`images/workbench8.bin`, on a FinalGROM 99 or in js99er) and its disk (`images/workbench.dsk`, in DSK1), on a TI with an F18A and a SAMS card. A PC is needed once, to make those two (they are prebuilt in the distribution); after that you can edit, assemble and run on the TI alone. WORKBENCH.md is the manual.

Two ways to work, marked throughout these documents.

Mark	Where	What you use
[PC]	on a PC with Python 3 and xdt99	any editor; `skbuild.py` builds a cartridge image; the simulator, js99er or a real TI runs it; `skdisk.py`, `skmusic.py` and the tests help
[TI]	on the TI-99/4A itself (or js99er playing one), with the workbench cartridge and its disk	the workbench's editor, assembler and launcher: nothing else
[PC or TI]	either	the same either way: the assembly source and every kernel call

A block is tagged when it belongs to one route only. Assembly source is **[PC or TI]** unless a note says otherwise (the **[TI]** route assembles one module of up to 4,096 bytes, at `>C000`: see the table below). Shell commands are **[PC]**; key sequences like "N, type a name, ENTER" are **[TI]**.

	[PC]	[TI]
Write the source	any editor	the workbench editor (80 columns, up to 800 lines)

Assemble	xas99: the full dialect, user macros, large programs	the workbench's native assembler: xas99's dialect without user macros (`.defm`) or forward `EQU`; image up to 4,096 bytes; symbols up to 12 characters
Modules	as many as you like (`app.json`); `GOMOD` and `.callmod` between them	one module; the only switch is back to the workbench (`GOMOD PG_WB`)
Data in SAMS pages (`data`), music compiled at build (`from`), `PD_` equates	yes	no
Your own font and colour schemes (`databank.a99`)	yes	no
Every kernel routine (`PUTS`, `VMODE`, `FOPEN`, `FSEL`, `JOYGET`, `PINIT`, `SNDTONE` ...)	yes	yes (`MUPLAY` needs a song in SAMS pages, so it is **[PC]**)
Output	a cartridge image (`NAME8.bin`) for js99er, FinalGROM or a real TI	the program runs inside the workbench; no cartridge (a TI cannot write a FinalGROM)
Run	simulator (`sk.py run`), js99er, real machine	`R` in the workbench, at once
Test	Python against the simulator: key scripts in, screen and memory out; the kernel's own test groups	run it and look; the `DBG` routine and screen breadcrumbs; `docs/HARDWARE-CHECKLIST.md`

1. Hello, kernel

[PC] Build and run the smallest example:

```
python3 tools/skbuild.py examples/hello
python3 tools/sim9900.py build/hello/hello8.bin --keys "<ALPHA>hello" --frames 200
```

[TI] The same program, on the TI: load `workbench8.bin` with `workbench.dsk` in DSK1, then press **N**, type `DSK1.HELLO/A99`, **ENTER** (TI file names have no dots: `hello.a99` is `HELLO/A99`), **A** (the screen says "Assembled: 194 bytes at >C000"), **R** (the program runs: type, and FCTN = quits). **E** shows the source in the editor. To type a program in yourself: **N**, `DSK1.MINE/A99`, **ENTER**, **E**, type it, **CTRL A** (saves and assembles), then **R**. (The anatomy below walks through `hello.a99`, line by line; WORKBENCH.md part 2a has a complete worked example, typed on the TI and built on the PC.)

The **[PC]** commands assemble the kernel, the loader and your module and writes `build/hello/hello8.bin`, a cartridge image. The second runs it in the simulator and prints the screen as text. To run it in js99er, load `hello8.bin` as a FinalGROM/cartridge image with the F18A and a SAMS card (512K or more) switched on in its settings.

Start your own application with the scaffolder, which writes a working one-module program:

```
python3 tools/newapp.py myapp --name "MY APP"
python3 tools/skbuild.py myapp
```

Anatomy of `examples/hello/hello.a99`

```
       COPY "kernel.equ"      the kernel's symbols (routines, variables, equates)
       COPY "macros.a99"      .enter, .leave, .push, .pop, .callmod
```

Every module begins with these two lines. `kernel.equ` is written by the build when it assembles the kernel: it holds the address of every kernel routine (`PUTS`, `KEYGET`, ...) and every equate (`COLS`, `RTEXT`, `APPRAM`, `KQUIT`, ...). Your module is assembled separately, loaded into SAMS pages, and never linked to the kernel: these symbols are the whole connection.

```
ATEXT  EQU  (RTEXT*16+RBG)*256        text on the page background
```

A cell's **attribute** is a byte: foreground colour *role* in the high nibble, background role in the low (part 5). It is passed in the *high byte* of a register, as every byte value in this kernel is, so the equate multiplies by 256.

```
       AORG >C000
       DATA INIT,TICK,REDRAW,0         the entry table
```

A module is assembled to run at `>C000` and starts with its **entry table**: four words, the addresses of `INIT`, `TICK`, `REDRAW` and a reserved word (part 3).

```
INIT   .enter
       BL   @REDRAW
       .leave
       RT
```

`INIT` is called when the kernel switches to your module. `.enter` and `.leave` save and restore the return address on the software stack: any routine that calls another needs them (part 4).

```
TICK   .enter
       MOV  R0,R0
       JEQ  TK9                        no key
       CI   R0,KQUIT*256               FCTN = ?
       ...
       SETO @QUITF                     ask the kernel to quit
```

`TICK` is called **once every frame** (sixtieth of a second) with `R0` holding the next key typed, in the high byte, or `0` for none. Setting `QUITF` non-zero asks the kernel to quit to the TI title screen. A key's code is a byte: ASCII for printable keys, with FCTN and CTRL chords in `>C0-FF` and `>80-BF` (part 6).

That is a whole program. Everything else in this guide is about doing more in `INIT` and `TICK`.

2. The machine, in one page

```
CPU address space             SAMS page(s)        what lives there
>0000-1FFF  console ROM       (not mapped)
>2000-3FFF  SLOT A            >02-03, fixed       kernel + application variables, the stack
>4000-5FFF  DSRs, mapper regs (not RAM)
>6000-7FFF  cartridge         (not mapped)        only the loader looks at it
>8300-83FF  scratchpad        (not mapped)        fast RAM: workspace, kernel hot variables
>8400-9FFF  I/O ports         (not RAM)
>A000-BFFF  THE KERNEL        >0A-0B, fixed       3.6K of 8K used
>C000-DFFF  SLOT C            your module (2 pages) the running module, 8K
>E000-FFFF  SLOT B            any 2 pages         your data: window >E000 and window >F000
```

- **Slot A** is where your *variables* live, always mapped: `APPRAM` (`>2300`) up to `APPTOP` (`>3F00`) is yours, 7,168 bytes; `MODRAM` (`>2200`, 256 bytes) is scratch for the running module and is not kept across a module switch. The stack is `>3F00-3FFF`.
- **Slot C** is where your *code* runs. Only one module is mapped at a time.
- **Slot B** is for *data* too big for slot A: you map one SAMS page at `>E000` and another at `>F000` as you need them (part 7).
- **VRAM** (16K, on the F18A): the screen's tables take `>0000-215F`; most of the rest is yours (part 5).

Register use, in one sentence: **R0-R9 are scratch, R10 is the stack pointer, R11 the return address, R12 the CRU base, R13-R15 are best left alone.**

3. The anatomy of a module

A module is up to 8,192 bytes of code and constant data assembled at `>C000`. The first four words:

Word	Name	Called	R0	R1
`>C000`	`INIT`	when the kernel switches to the module (boot, or `GOMOD`)	the page of the module before, 0 at boot	the argument `GOMOD` was given, 0 at boot
`>C002`	`TICK`	every frame	the next key code (high byte), or 0	
`>C004`	`REDRAW`	**never by the kernel**; by convention, your module's "draw the whole screen"		
`>C006`	reserved			
`>C008...`	your entries	by `MODCALL` from another module		

INIT sets the module up and draws. It runs with the screen already in Text 80 and colour scheme 0 (or whatever the previous module left), and with whatever the previous module drew still on the screen: clear it (`CLS`) if you do not want it.

Slot A variables are as the previous module left them: *only the first module's `INIT` can count on zeros*; later `INIT`s must initialise what they use.

TICK does the work. It must return reasonably quickly (part 10): while it runs the keyboard is not polled, the sound list does not advance and the screen does not change. A `TICK` that has nothing to do should simply return. Setting `QUITF` ends the program after `TICK` returns; so does `INIT`. `GOMOD` (part 8) likewise takes effect after the routine that called it returns.

REDRAW is yours to use. The usual pattern is that `INIT` ends with `BL @REDRAW`, and other modules call it (through `MODCALL`) to put your screen back after a pop-up covered it.

A module may be **up to 8K**, but not a byte past it: the build stops with an error. Anything that does not fit is a second module (part 8), or data in a SAMS page (part 7).

The kernel needs nothing else from a module. Everything not in the entry table is yours to name, except for the names `kernel.equ` exports: the **public routines** (`kernel/public.txt`, documented in ROUTINES.md) and every **equate** in `kdefs.a99`. A clash with one of those is an assembly error ("duplicate symbol"): easy to see. The kernel's other labels are private and never reach your module, so a new kernel release cannot collide with your labels.

4. Calling conventions and registers

The stack. `R10` is a software stack that grows down from `STKTOP` (`>4000`) in slot A, 256 bytes (128 words). Only four macros touch it:

Macro	Does
`.enter`	push `R11` (the return address)
`.leave`	pop `R11` (then you `RT`)
`.push Rn`	push a register
`.pop Rn`	pop it back

A routine that calls another (`BL`) must begin with `.enter` and end with `.leave` then `RT`, because `BL` overwrites `R11`. A **leaf** (a routine that calls nothing) needs neither and may simply `RT`. A leaf may also *tail-call* with `B @OTHER`: `OTHER`'s `RT` then returns to the leaf's caller.

Flags do not survive `.leave` (it is a `MOV`, which sets the flags from `R11`). A routine that ends with `.leave` returns its result in a register; the caller tests that register (`MOV R0,R0` then `JEQ`). Leaves may return results in the flags, but it is simpler not to.

Registers.

Register	Rule
R0-R9	scratch. *Every routine's header says which it changes ("Uses R0-R7").*
R10	the stack pointer: only the macros change it
R11	the return address; `BL` overwrites it
R12	the CRU base: every user of the CRU loads it first; nothing relies on it across a call
R13-R15	the TMS9900 context-switch registers in the console's interrupt routine, and the GPL workspace's in DSRs: leave them alone unless you know why

The rule that matters most: a value you still need after a call goes in a register the callee does not use, or in memory. Read the callee's "Uses". The common trap: `PUTS` changes `R0-R7`; `DECW` changes `R0-R8`; a loop counter kept in `R7` across a `PUTS` is gone. Loop state belongs in `R8-R9` (with `DECW`, only `R9`) or in a slot A variable.

`R0` can never be an index register (`MOV @TABLE(R0),R1` does not assemble). `MOVB` sets only the high byte: after `MOVB @X,R0` the low byte is whatever it was, so `CLR R0` first if you then compare the whole word. `SLA Rn,0` shifts by `R0`'s low four bits, not zero. `xas99` evaluates expressions strictly left to right (`A+2*3` is `(A+2)*3`): parenthesise.

5. The screen

The screen is 80 columns by 30 rows. A **position** is `row * 80 + column` (0-2399). Each cell has a **glyph** (a byte, the name table at VRAM `>0000`) and an **attribute** (a byte, VRAM `>1800`): the foreground role in the high nibble, the background role

in the low. Every routine that writes text takes the attribute in the high byte of a register, and glyphs likewise.

Colour roles and schemes

Palette entries 1-15 are *roles*, not colours (`kdefs.a99`):

Role	Name	Role	Name
1	`RBG` page background	9	`RSELBG` selection background
2	`RTEXT` text	10	`RSELFG` selection foreground
3	`RBOLD` emphasis	11	`RBARBG` status bar background
4	`RITAL` italic	12	`RBARFG` status bar foreground
5	`RBOLIT` bold italic	13	`RRULER` rules and frames
6	`RSCRPT` super/subscript	14	`RMARK` markers, labels
7	`RDIM` dim	15	`RACCNT` accent: the cursor
8	`RERROR` error		

An attribute names two roles: `(RTEXT*16+RBG)*256` is text on the page. Entry 0 is transparent (the backdrop shows) so attributes use 1-15 only. A **colour scheme** is 16 palette words: `SETSCH` (R0 = 0-4) loads one, and the whole screen changes colour without a cell being rewritten. The five built in are TI-Writer's Screen Color order: blue/white (default), green/black, green/white, light blue/black, black/white. Your own schemes: part 12.

Think of roles as a theme: say what a cell *means* (error, cursor, status bar), not what colour it is. The roles are named for a text editor, but nothing in the kernel depends on the names; use them as you like.

Writing text

Routine	Does
`PUTS`	R0 = position, R1 = a *length-prefixed* string, R2 = attribute. Uses R0-R7
`PUTC`	R0 = position, R1 = glyph (high byte), R2 = attribute
`FILLN`	R0 = position, R1 = glyph (high byte), R2 = count: glyphs only
`FILLA`	R0 = position, R1 = attribute (high byte), R2 = count: attributes only
`DECW`	R0 = position, R1 = unsigned number, R2 = attribute; returns R3 = digits
`HEX2` / `HEX4`	R0 = position, R1 = number, R2 = attribute: two / four hex digits
`CLS`	R1 = attribute: every cell a space in it
`BOX`	R0 = top-left position, R1 = interior width, R2 = interior height, R3 = attribute: a frame

Strings are length-prefixed: `STRI 'text'` writes the length byte then the characters, so `LI R1,TITLE` / `BL @PUTS` just works. If you write them by hand with `BYTE n` then `TEXT`, count carefully (a miscounted string is the commonest bug) and put `EVEN` after any byte data before the next code label, or the label lands on an odd address where the TMS9900 silently executes from the even address below.

Positions past the end of a row simply continue on the next; nothing clips.

Glyphs

The pattern table holds 256 glyphs of 6 x 8 pixels (Text 80 shows the left six bits of each byte): UI glyphs 0-31 (box lines, arrows, markers, shades), ASCII 32-126, 127 the "Latin-1" marker, **underlined ASCII 128-222 (code + 96)**, more UI glyphs 223-255. The chart is GLYPHS.md; `examples/diag` draws it on screen (`>00..>E0`). To underline a character, add 96 to its code. The font is generated by `tools/font.py`; to use another, give your application its own `databank.a99` (part 12).

VRAM

The standard map, which every mode but Graphics II shares (VIDEO-MODES.md part 3 has the details and Graphics II's map):

Range	Holds

`>0000-095F`	name table (2,400 bytes at 80 x 30)
`>0960-097F`	the GPU's mailbox and program (`GPUMB`, `GPUAD`)
`>09C0-09DF`	Graphics I's colour table (`GM1CT`)
`>0A00-135F`	the attribute table (`ATRTAB`, 2,400 bytes at 80 x 30)
`>1400-14FF`	sprite attribute table (`SATAB`)
`>1500-17FF`	**free** (768 bytes)
`>1800-1FFF`	pattern table, plane 0: the 256 glyphs (`PATTAB`)
`>2000-2FFF`	**free**, or the extra pattern planes of enhanced colour 2 and 3
`>3000-35FF`	**free**, or the sprite patterns (`SPTAB`)
`>3600-36FF`	the file service's record buffer
`>3700-37D7`	the file service's PAB
`>37D8-3FFF`	the disk controller's own: **never touch**

Write to VRAM with `VMBW` (block), `VFILL`, `VSBW` (one byte) and read with `VMBR`, `VSBR`. Each sets the VDP address and then moves bytes; the address auto-increments. **Interrupts are masked (`LIMI 0`) everywhere except inside `VSYNC`**, because an interrupt between the two bytes of an address write corrupts it; your code, in `TICK`, always runs with interrupts masked, and you leave it that way.

Moving rows: `VSHIFT`

`VSHIFT` copies R2 rows of `TXCOLS` bytes (80 in the boot mode) from VRAM address R0 to R1 using the F18A's DMA engine, started by a tiny GPU program the kernel installs. Scrolling a region of the screen is two `VSHIFT`s (name table and attribute table) and a redraw of the exposed row. R3 = 0 copies forwards, 2 backwards (for overlapping moves; then R0 and R1 point at the *last* byte of each block). It waits for the GPU to finish. Not every F18A firmware has the DMA engine: the kernel tests it at boot and sets `NODMA` non-zero when it does not; code that scrolls should redraw rows itself in that case.

Other display modes

[TI] `programs/mode.a99` on the workbench disk switches between Text 80 and Text 40 with `VMODE` and redraws with positions computed from `TXCOLS` (assemble it, run it, press 1 or 2; FCTN 9 returns). Only those two modes are tested on the TI route. **[PC]** `examples/modes` shows all fifteen screens, with the pictures in VIDEO-MODES.md. The source of either is the same kind of code: `LI R0,VMT40 / LI R1,VO30+VOPOS / BL @VMODE`.

`VMODE` switches the F18A to any of its modes: R0 = the base mode (`VMT80`, `VMT40`, `VMGM1` Graphics I, `VMGM2` Graphics II, `VMMC` Multicolor), R1 = the options added together (`VO30` 30 rows, `VOPOS` a colour attribute per position, `VOTE1-3` / `VOSE1-3` enhanced colour for tiles / sprites, `VOSPR` sprites on, `VOS16`, `VOSMG`):

```
        LI   R0,VMGM1              Graphics I, 8-colour tiles (enhanced colour 3), an attribute per position
        LI   R1,VOPOS+VOTE3
        BL   @VMODE
```

It re-initialises the registers and the VRAM tables (a blank screen, the glyphs, the colour scheme) and sets `TXCOLS`, `TXROWS`, `TXCELLS`, `TXATTR` and `NAMBAS`, which the text routines follow; **everything on the screen is gone afterwards**. `VREG` (R0 = register, R1 = value in the high byte) sets the rest: tile layer 2, scrolling, the bitmap layer, the sprite limits. The text routines work in Text 80, Text 40 and Graphics I; in Graphics II, Multicolor and the sprite and layer work you write the VRAM yourself. Two things trip people up: in an enhanced colour mode **the attribute byte is flags** (priority, flips, transparency, palette select), not a pair of colour roles; and **Graphics II uses the VRAM the file service needs**, so do not read or write files while it shows. `examples/modes` has a screen for every mode and VIDEO-MODES.md explains each, with pictures.

The cursor

There is no hardware cursor: draw one by giving a cell the accent attribute `(RBG*16+RACCNT)*256` (`ACUR` in `diag.a99`) and putting the old cell's attribute back when it moves.

6. The keyboard

[PC or TI] The code is the same on both routes. **[TI]** `hello` (echoes keys), `examples/joystick/joy.a99` (key codes and both joysticks) and `programs/beep.a99` (keys 1-8 play notes) are on the workbench disk.

A **key code** is one byte. The kernel decodes the whole chord (key + modifiers) to a single code, so your program never tracks SHIFT or CTRL itself:

Codes	Meaning
`>20-7E`	printable characters, as typed (SHIFT and the characters FCTN shows are included)
`>0D`	ENTER (with any modifier but CTRL)
`>80-BF`	CTRL + key: `>80 OR (the key's code AND >3F)`: CTRL A = `>81` ... CTRL Z = `>9A`, CTRL 0-9 = `>B0-B9`, CTRL ENTER = `>8D`
`>C0-FF`	FCTN + a key that has no character of its own: FCTN E/S/D/X (the arrows) = `KUP KLEFT KRIGHT KDOWN` = `>C5 >D3 >C4 >D8`; FCTN 1-9 = `>F1-F9` (DEL, INS, ERASE, CLEAR, BEGIN, PROC'D, AID, REDO, BACK; `KDEL KINS ...` in `kdefs.a99`); FCTN 0 = `>F0`; FCTN = = `KQUIT` = `>FD`

FCTN takes precedence over CTRL over SHIFT. ALPHA LOCK capitalises letters typed with no modifier. The complete table is in HARDWARE.md.

A code is produced **when a key goes down**, with the modifiers held then; releasing a modifier before the key changes nothing; holding a key repeats it after 24 frames and then every 4 (400 ms, then 15 a second). Codes go into a 64-entry buffer.

Getting keys. `TICK` is handed one key per frame, in `R0` (high byte). To take more (a batch of typing), call `KEYGET` yourself: it returns the next code in `R0`, or 0. To wait for a key in a moment that really is modal ("Press any key"), `KWAIT` runs frames until one arrives; `KFLUSH` throws away what is buffered. Compare with high-byte constants: `CI R0,KQUIT*256`, `CI R0,>2000`, `CI R0,>9400` (CTRL T).

Joysticks. Both joysticks are read every frame, with the keyboard (they are keyboard columns 6 and 7 on the TI). `JOYGET` takes R0 = 1 or 2 and returns R1 = what is down now and R2 = what went down since the last `JOYGET`, as bits `JFIRE JLEFT JRIGHT JDOWN JUP` (1 = active), so a program sees every press however brief:

```
       LI   R0,1              joystick 1
       BL   @JOYGET           R1 = down now, R2 = new presses
       COC  @JBIT,R2          a fire press since I looked?   (JBIT DATA JFIRE)
       JEQ  SHOOT
       MOV  R1,R3
       ANDI R3,JLEFT          held left?
       JNE  GOLEFT
```

`JOY1` and `JOY2` hold the states themselves, if you would rather read a word. To drive a menu or editor written for keys with a joystick, set `JOYKEY` (1 = joystick 1, 2 = joystick 2, 3 = both): the kernel then types `KUP KDOWN KLEFT KRIGHT` and `KENTER` codes into the key buffer for it, with the keyboard's repeat (24 frames, then every 4), and your `TICK` sees them as keys. `examples/joystick` shows all of it. A joystick that is not plugged in reads as nothing down; there is no presence test. Not tried on real joysticks.

Modifiers separately. `KMODS` holds the modifiers of the last code (`>80` FCTN, `>40` SHIFT, `>20` CTRL, `>10` ALPHA LOCK) in the high byte, and `ALOCK` is non-zero while ALPHA LOCK is down.

The console's own QUIT (FCTN =) is *disabled*: the kernel sets the console interrupt routine's flag byte to `>50`, so FCTN = reaches your program as code `KQUIT` like any key, and your program decides what it means (ask "Save first?", then set `QUITF`). The diag and hello examples quit immediately.

Every key restarts the console's nine-minute screen-blank timer, and turns the display back on if it had blanked.

7. Memory

Variables in slot A

Declare every variable once, as an offset from `APPRAM` (see `examples/browser/shared.a99`):

```
NFILES EQU  APPRAM+0       files found
SEL    EQU  APPRAM+2       the selected one
```

and use them with `@`: `MOV @SEL,R1`. There are 7,168 bytes between `APPRAM` and `APPTOP`; modules share state only through addresses like these, agreed in a file that every module `COPY`s (`shared.a99`). Slot A is cleared by the loader, so the first

module's `INIT` can rely on zeros.

`MODRAM` (`>2200`, 256 bytes) is the one scratch area the kernel names for you, for a module that wants private work space: *it is not kept across a module switch* (and the self-test uses it for its results). For state that must survive a trip to another module use `APPRAM`.

The scratchpad (`>8300-83FF`) is the fast RAM, and the kernel and the console use nearly all of it (map in ARCHITECTURE.md). The one block that is yours is **`>8380-83BF`** (64 bytes: the loader uses it only while it copies the cartridge): good for a hot loop's counters or a small table.

Writing `>2000-3FFF` is always safe and always mapped. Never put the stack or a variable in `>A000-FFFF`: those windows are remapped.

SAMS pages for big data

[TI] `programs/paging.a99` on the workbench disk takes four pages from the pool, writes each page's own number into it through slot B, checks them, and puts slot B back with `GETPB`/`SETPB`: this part's code, running. **[PC]** the same code builds with `skbuild.py` (change the return to the workbench, which exists only on the TI route). Data files that the build puts into pages for you (`data` in `app.json`, the `PD_` equates) are **[PC]** only.

The SAMS card gives you 4K pages, numbered from 0 (128 of them on a 512K card, 256 on 1M, up to 4,096 on a 16M card). The kernel and the build own some; the rest are yours:

Pages	Use
`>00-01`	never used (the console's windows)
`>02-03`	slot A
`>04-09`	**free** (24K): the CPU never maps them, so only slot B/C can show them; the loader does not test them
`>0A-0B`	the kernel
`>0C-0F`	slot C's and B's default pairs
`>10-11`	the data bank (glyphs, colour schemes)
`>12...`	your modules, two pages each, in the order `app.json` lists them (or where you say)
after the modules	`PGFREE` and up: yours, and the **page pool's** natural range

`MAPE` maps one page into window `>E000`, `MAPF` into `>F000`; afterwards `@>E000` is that page's first byte. `PAGEB` maps two *consecutive* pages into both. All of them keep `CURBE`/`CURBF` (the pages slot B holds) in step, **which is what makes it safe to borrow slot B**. A page is a number 0-4095 (a card of up to 16M):

```
       BL   @GETPB            R0, R1: the pages mapped in >E000 and >F000 now
       .push R0
       .push R1
       LI   R0,MYPAGE
       BL   @MAPE
       ... use @>E000 ...
       .pop R1
       .pop R0
       BL   @SETPB            put both windows back as they were
```

(Before 1.4.0 the saved value was one packed word, `MOV @CURPB,R0 ... BL @SETPB`; it is now two pages, so a page above 255 fits.)

Never write a mapper register any other way (`MAPPG` does not record the pages: it is the low-level routine). **Code never remaps the window it is running in.** Code in slot C may freely remap slot B; nothing but the kernel's main loop remaps slot C, and nothing remaps slot A. A buffer you hand to a routine must not be in `>E000-FFFF` if the routine itself remaps slot B (the rule applies to your own routines too).

The page pool

`PINIT` (R0 = first page, R1 = the page after the last) declares a pool and frees all its pages. `PALLOC` returns the lowest free page in R0 (0 when none is left); `PRELS` gives one back (twice is harmless; a page outside the pool is ignored). A page's contents are never touched. The pool is a bitmap (512 bytes at `>BE00`, in the kernel's pages) for up to 4,096 pages; `FREECT` is the free count. The viewer in `examples/browser` takes pages from `PGFREE`, fills them with lines of a file, and draws from them:

```
       LI   R0,PGFREE
       LI   R1,PGFREE+8       an 8-page pool
       BL   @PINIT
       ...
       BL   @PALLOC           R0 = a page
       BL   @MAPE             map it at >E000
```

Check `NPAGES` (128 or 256) if you take pages near the top: the card the user owns may be bigger than `min_pages`, never smaller (the loader refuses it).

8. Several modules

[PC] only. The TI route has one module: the workbench runs your program as a module and the only switch it offers is back (`LI R0,PG_WB / CLR R1 / BL @GOMOD`). Everything in this part is for programs built with `skbuild.py` from an `app.json` that lists more than one.

List them in `app.json`; the build assigns pages and writes their names as equates:

```
"modules": [
  {"name": "browse", "source": "browse.a99"},
  {"name": "view",   "source": "view.a99"}
]
```

gives `PG_BROWSE` and `PG_VIEW` (the first page of each: names upper-cased) in every module that includes `kernel.equ`. `"boot"` names the module that starts (default: the first).

Switching: `GOMOD`

```
       LI   R0,PG_VIEW        the module to switch to
       MOV  @SEL,R1           an argument for its INIT
       BL   @GOMOD
```

`GOMOD` only *records* the request; the kernel switches when your `TICK` (or `INIT`) returns, so finish what you are doing first. The new module's `INIT` is called with `R0` = your page and `R1` = the argument; **slot A is unchanged,** `MODRAM` **is not preserved, the screen is as you left it.** `PREVPG` holds the module that ran before the most recent switch, so a module can go "back": `MOV @PREVPG,R0 / CLR R1 / BL @GOMOD`. Switching is not a call: it does not return to the old module; the old module's `INIT` runs again if you go back (so `INIT` must be able to redraw from your slot A state).

Calling: `MODCALL`

Sometimes you want a *subroutine* in another module and then to carry on here. Give the other module an extra entry (words 4 and up in its table; word 3 is reserved):

```
       AORG >C000
       DATA INIT,TICK,REDRAW,0,PING         entry 4 is PING

PING   LI   R0,>1234
       RT
```

and call it from another module with the `.callmod` macro:

```
       .callmod PG_DIAG,4        R0 = the page, R1 = the entry's address, BL @MODCALL
```

`MODCALL` maps that module into slot C, calls the routine, maps *your* module back and returns, with `R0` as the routine left it. The mapping itself changes `R0-R4` and `R7`, so pass arguments and take extra results in **`R5, R6, R8, R9`** (they cross the call untouched) or in slot A. Calls nest. The called routine runs with *its* module in slot C, so it may not touch your code or data in slot C.

Rules

- A module cannot call another's routines directly (they are not mapped): `MODCALL` or `GOMOD`.
- Shared constants and addresses go in one `shared.a99` that each module `COPY`s (see `examples/browser/shared.a99`).
- Labels in different modules are independent: two modules may both have a `LOOP1`. Only the names `kernel.equ` exports (the public routines and the equates) are reserved.
- A module that must be callable by `MODCALL` while *also* being switched to needs its entry table the same either way.

9. Files

[TI] `programs/copy.a99` on the workbench disk copies `DSK1.SRC` to `DSK1.DST` through two handles at once: the file calls below, running. **[PC]** `tests/apps/files2` is the same idea as a cartridge, checked against TI's own disk controller ROM in the simulator. **[PC or TI]** The code is identical.

The file service speaks to any device through the console's device service routines: the disk controller (`DSK1.`), TIPI (`PI.`), a printer port (`PIO`), and so on. It can hold up to three files open at once (part 9, "Several files").

Routine	In	Out
`FOPEN`	R0 high byte = flags; R1 = record length; R2 = the name, length-prefixed	R0 = 0 or the error
`FREAD`	R1 = buffer (up to 255 bytes)	R0 = 0, 5 at end of file, or the error; R2 = the record's length
`FWRITE`	R1 = buffer; R2 = length	R0 = 0 or the error
`FCLOSE`		R0 = 0 or the error
`FDELETE`	R2 = the name	R0 = 0 or the error
`FCNUM`	R1 = a catalog record, R2 = n (0-2)	R0 = number n of the record

Flags are added together, one from each group, and passed in the *high byte* of R0:

Group	Choices
record contents	`FDISP` (DISPLAY, default) / `FINTL` (INTERNAL)
record length	`FFIXED` (default) / `FVAR` (VARIABLE)
mode	`FIN` input, `FOUT` output (creates the file), `FUPDT` update, `FAPP` append

So a DIS/VAR 80 text file for reading is `LI R0,FIN+FVAR` with `R1 = 80`:

```
       LI   R2,NAME            NAME STRI 'DSK1.NOTES'
       LI   R0,FIN+FVAR
       LI   R1,80
       BL   @FOPEN
       MOV  R0,R0
       JNE  FAILED
LOOP   LI   R1,MYBUF           up to 255 bytes
       BL   @FREAD
       MOV  R0,R0
       JNE  DONE              5 = the end, anything else an error
       ... R2 = the line's length ...
       JMP  LOOP
DONE   BL   @FCLOSE
```

Writing is the same with `FOUT+FVAR` and `FWRITE`. **Results** (R0): 0 worked; 1-7 the DSR's error (5 beyond the end of the file, 2 bad open attributes, 7 a file error, ...); `>FF` no card has that device, or the name was longer than 53 characters. Two behaviours seen with TI's own disk controller (run in the simulator against its real DSR) that the stand-in disk does not show:

- **After the read that reports 5 (end of file) the file is already closed**; `FCLOSE` then answers 7. Treat 5 as "done" and ignore `FCLOSE`'s result after it (call it anyway: other devices may need it).
- **Opening a file that does not exist for input answers 2**, where the stand-in says 7. Do not tell "missing" from other open failures by the exact code; any non-zero result means "could not open".

The catalog. Opening a name that is only a device and a dot (`DSK1.`) as `FIN+FINTL` (INTERNAL, FIXED) with record length 38 reads the disk's catalog: the first record describes the disk, then one record per file, then an empty name. A record is a length-prefixed name, then three radix-100 numbers (`FCNUM` decodes small integers): for a file, its type (1 DIS/FIX, 2 DIS/VAR, 3 INT/FIX, 4 INT/VAR, 5 program; negative if protected), its size in sectors and its record length; for the disk, 0, its sectors and the sectors free. `examples/browser/browse.a99` shows it all.

What the service looks after for you (the rules the Lantern Coast learned the hard way): the GPL workspace, the name pointers, saving and restoring the scratchpad the DSR may change (except `>8370`, which must be what the controller left it), the controller's one-time power-up routine, the PAB in VRAM, and clearing the previous call's error bits.

What you must remember:

- A DSR call changes `R6-R9` along with `R0-R5`: keep nothing you need in them, or push it.
- It takes from milliseconds to seconds; call `KIDLE` between records if keys must not be lost (the viewer does).
- Interrupts are off during the call.
- The VRAM from `>3600` is the service's and the controller's; keep your own VRAM below it.
- Up to three files are open at once (below); open a fourth only after closing one.

Several files

Each open file has a **handle**, 0-2. `FSEL` (R0 = the handle) says which handle `FOPEN`, `FREAD`, `FWRITE`, `FCLOSE` and `FDELETE` act on; it is 0 until you change it, so a program that never calls `FSEL` works with one file, as before. Every handle keeps its own file open and its own place in it while you use the others:

```
       CLR  R0
       BL   @FSEL             handle 0: the input
       LI   R2,SRC
       LI   R0,FIN+FVAR
       LI   R1,80
       BL   @FOPEN
       LI   R0,1
       BL   @FSEL             handle 1: the output
       LI   R2,DST
       LI   R0,FOUT+FVAR
       LI   R1,80
       BL   @FOPEN
LOOP   CLR  R0
       BL   @FSEL
       LI   R1,BUF
       BL   @FREAD            a record from handle 0 ...
       MOV  R0,R0
       JNE  DONE
       LI   R0,1
       BL   @FSEL
       LI   R1,BUF
       BL   @FWRITE           ... written to handle 1 (R2 = the length FREAD gave)
       JMP  LOOP
```

`FSEL` answers `>FF` for a handle above 2 (and keeps the old choice). The limit is the disk controller's: it keeps three file buffers, so a fourth open fails with its error; a printer or another device's file counts as one of the three. Each handle has its own PAB in VRAM (`>3700`, `>3740`, `>3780`: the controller tells open files apart by their PABs) and they share one record buffer, since a call finishes with it. `FDELETE` uses the selected handle's PAB, so close that handle's file first. `tests/apps/files2` is a working copy-while-reading example, checked against TI's own controller ROM.

10. Time

The main loop runs: *wait for the frame, poll the keyboard, take one key, call `TICK`*. One pass is one frame, a sixtieth of a second (a fiftieth on a PAL console). `TICK` has the whole frame to itself: the VDP's interrupt is masked while it runs, so the console's interrupt routine (which advances the frame counter, plays sound lists and blanks the screen) does not run. If `TICK` takes longer than a frame, the lost frames are not made up: the next pass starts at once. `FRAME` counts main-loop passes (and the frames `KIDLE` sees), so it is a *tick* count, not exactly time.

Long work. If a task takes more than a few frames (reading a file, sorting, drawing a page), poll the keyboard between steps so typed keys are not lost:

- `KIDLE` checks the VDP's frame flag and, if a frame has passed, polls. Call it between VDP transfers, never in the middle of one. It uses `R0-R7` and `R12`.
- Inside a tight byte loop, read `VDPSTA` yourself and call `KPFR` when its top bit is set; `KPFR` keeps every register but `R0`.

Keys land in the buffer; take them with `KEYGET` when the work is done (or *flush* them if the user should not be able to queue them).

Frame timing. `VSYNC` waits for the next frame and is the only place interrupts are allowed. To wait N frames: loop `BL @VSYNC` N times (or `BL @KWAIT` for a key).

Sound and music. `MUPLAY` starts a song held in SAMS pages and the main loop steps it every frame (and `KIDLE`, `KPFR` and `KWAIT` do too, so music goes on through long operations that poll the keyboard). `MUSTOP`, `MUPAUSE`, `SNDTONE` and `SNDOFF` stop, pause, make a tone directly, and silence the chip. Songs are compiled on the PC from a MIDI file or a text score (`tools/skmusic.py`), and the build puts them in pages (`"from"` in `app.json`'s `data`); MUSIC.md has the whole story, the score

language and the song format. The console's own sound lists (the interrupt routine plays them from VRAM or GROM) still work and share the chip with the player; the two have not been mixed in a test.

Screen blank. After about nine idle minutes the console blanks the display (it writes VR1 from `VR1CPY` with the display bit cleared); any key restores it (`KPUT` writes `VR1CPY` back with the display bit set). If you change video mode, keep `VR1CPY` (`>83D4`) equal to the VR1 value that mode runs with (Text 80: `>F0`), display bit set, or the screen will come back in the wrong mode.

11. Quitting

Set `QUITF` non-zero (`SETO @QUITF`) from `TICK` or `INIT`. After the routine returns the kernel (`KEXIT`) silences the sound chip, waits up to a second for the keys to be released (so the console does not see the same FCTN = and quit its own menu at once), restores the F18A to its power-up state (30 rows, position attributes, the stop-sprite register, the 16 standard colours), switches the SAMS mapping off, selects cartridge bank 0 (so the console's menu finds your program) and resets the console, exactly as FCTN QUIT does. Close any open file first; there is no exit hook.

12. Building

[PC] This part is the PC route: `skbuild.py` and `app.json`. **[TI]** The workbench builds for you, with none of this: one module, `AORG >C000`, an entry table first, `COPY "kernel.equ"` (the file KERNEL/EQU on the disk); **A** assembles it and **R** runs it. There is no `app.json`, no `data`, no second module and no cartridge image on the TI route (WORKBENCH.md part 5).

`python3 tools/skbuild.py APPDIR` reads `APPDIR/app.json`:

```
        {
          "name": "MY APP",             menu name and refusal screens (20 characters)
          "min_pages": 128,             SAMS pages required: 32, 64, 128 ... 4096
          "modules": [
            {"name": "main",  "source": "main.a99"},
            {"name": "other", "source": "other.a99", "page": "0x20"}
          ],
          "boot": "main",
          "data": [ {"file": "words.bin", "page": "0x40"} ]
        }
```

- **`min_pages`** is what your program needs; a smaller card is refused on a screen that names the app and the size ("NEEDS A SAMS MEMORY CARD WITH 512K").
- **Module pages** are assigned from `>12`, two each, unless you give `"page"` (an even start is conventional; allowed: `>04-09` and `>12` up). Overlaps, pages past `min_pages` and the kernel's own pages are errors.
- **`data`** files are copied into SAMS pages at start-up, in 8K pieces to consecutive page pairs: a dictionary, a map, a table. The equate `PD_<FILE>` gives the first page. The image grows with them (8K per piece), so the cartridge's size is the limit; Press's 512K image has run in js99er but, at the time of writing, not on a real FinalGROM 99.
- **`PGFREE`** (equate) is the first page the build leaves unassigned: where your pool starts.
- The output `build/APPDIR/NAME8.bin` is the cartridge image: 8K banks, padded to a power of two, bank 0 the loader. A real FinalGROM 99 and js99er take it as is.
- **Listings** (`.lst`) of every bank are kept beside it; `kernel.equ` is the symbol table.
- **Your own font and colour schemes:** put a `databank.a99` in your application's directory (copy `kernel/databank.a99` and edit): the build looks there first. Keep its layout: 256 glyphs at `>E000`, five schemes of 16 words at `>E800`.

The build runs `tools/font.py` and `tools/keytab.py` first (they write `gen/font.a99` and `gen/keytab.a99`: generated, never edit them).

13. Testing

[PC] You can run and test an application without a TI (this part). **[TI]** On the TI you test by running it and looking: put `COPY "dbg.a99"` in your module and `BL @DBG` anywhere, to show R0-R3 and wait for a key (every register kept), draw a heartbeat (`@FRAME` in a corner), and work through `docs/HARDWARE-CHECKLIST.md`. The assembler itself is tested on the PC, against xas99, in the simulator.

```
python3 tools/sim9900.py build/myapp/myapp8.bin --keys "<ALPHA>hello<ENTER>" --frames 300
```

prints the screen as text. In Python (see `tests/test_kernel.py`):

```
import sim9900
cart = open('build/myapp/myapp8.bin', 'rb').read()
m, cpu, halt = sim9900.run(cart, keys='<C-t><WAIT 600>', frames=1000)
print(sim9900.screen(m))                # 30 lines of 80 characters
m.rw(0x2300)                             # a word of memory (a variable)
m.vram[0x1000:0x1008]                    # VRAM
m.palette, m.vregs                       # the F18A's palette and registers
```

Key scripts: plain characters type themselves (capitals with SHIFT), `<ENTER>`, `<SPACE>`, `<C-x>` CTRL + x, `<F-x>` FCTN + x (`<F-=>` quit, `<F-e>` up, `<F-x>` down, `<F-s>` `<F-d>` left and right), `<S-x>` SHIFT, `<ALPHA>` toggles ALPHA LOCK (**it starts down**, as in js99er, so type `<ALPHA>` first for lower case), `<WAIT n>` waits n frames, `<HOLD n>` makes the next key a hold of n frames. The first key goes down at frame 60.

Pictures. `tools/f18render.py` draws any F18A mode from the simulator's state: `f18render.render(m)` returns a numpy picture (256 x 192, 240 rows, or 512 wide in Text 80) and `f18render.save(m, 'shot.png')` writes it; compare a pixel with `f18render.palette_rgb(m)[entry]`. `tests/test_kernel.py modes` does that for every screen of `examples/modes`.

Hooks and disks. Pass `setup=fn(m)` to prepare the machine before it runs: `m.dvfiles['NAME'] = [b'record', ...]` puts a file on the simulator's DSK1 (all DIS/VAR 80 unless `m.ftypes['NAME'] = (variable?, internal?, record length)` says otherwise), `m.bad_pages.add(0x2A)` makes a page fail its test, `run(..., pages=64)`, `sams=False`, `f18a=False` try other machines. After a quit, `m.at_reset` holds the F18A registers, palette and mapping at the console reset.

Test what a person would see: read the whole screen, not a line; give a run enough frames to settle; check the cursor and the data, not only the first key. `tests/test_kernel.py` is the model (the groups `boot refusal ram keys quit modules files browser rom`).

The simulator is not the machine. It models the TMS9900, the VDP and the F18A's text modes and DMA, the SAMS card, the keyboard matrix and a stand-in disk controller. It does not model timing, and its stand-in disk is kinder than a real controller (see the two differences in part 9). **Real disk controller:** with `TIFDC_ROM=path` to the controller's 8K ROM (join the two 4K EPROM images, low half first) tests can give the machine a `tifdc.TIFDC(image=...)` in `setup` (see the `realdsr` group): TI's own DSR then runs on an emulated WD1771 and a 90K disk image, as in js99er. Test on js99er and on real hardware before you believe it (part 14). **Real console ROM:** if you have your own copy of the TI console ROM (8K, raw or as an EA5 image) set `TI_CONSOLE_ROM=path` (or pass `rom=`): the simulator then runs TI's real interrupt routine instead of a model of it, which catches mistakes in how a program treats the console's variables. The ROM is TI's: it is never part of this repository; keep it outside.

The workbench. WORKBENCH.md: an editor, an assembler and a launcher that run on the TI, so a one-module program can be written, assembled and run there.

Developing with a real TI. DEVELOPING-ON-A-TI.md: the first-run ladder, what each loader refusal means, debugging with no debugger (`dbg.a99`), editing sources on the TI, and what it would take to build there as well.

The self-test on hardware. `examples/diag` is meant to be the first thing run on a new machine: press CTRL T for the kernel's self-test (hardware, paging, pool, module call, keyboard, screen, DMA, a DSK1 write/read/delete), CTRL 3 to cycle colour schemes, every key shows its code.

14. The checklist: what has already cost a day

1. **Registers across calls.** Check each callee's "Uses" (`PUTS` R0-R7, `DECW` R0-R8, `KIDLE` R0-R7 and R12, a DSR call R0-R9). Keep loop state in memory.
2. `.leave` **clobbers the flags.** Test a result register after the call.
3. `R0` **is not an index register;** `MOVB` **leaves the low byte;** `SLA Rn,0` **shifts by R0.**
4. `EVEN` **after byte data** (or a code label lands on an odd address); count `TEXT` lengths or use `STRI`.
5. **Jumps reach about 128 words**: when one goes out of range, invert the condition around a `B @label`.
6. **Expressions evaluate left to right**: parenthesise.
7. **Code never remaps the window it is running in**; slot C belongs to the kernel's main loop.
8. **Map slot B only with** `MAPE/MAPF/PAGEB/SETPB`, so `CURBE/CURBF` are true; save them with `GETPB` and restore them with `SETPB` when you borrow.
9. **Interrupts are masked outside** `VSYNC`**.** Do not `LIMI 2` in your code; do not poll the keyboard in the middle of a VDP transfer.
10. **Never write an F18A register you do not need**, and do not unlock the F18A twice (the second unlock locks it): the kernel handled both.
11. **Power-up RAM is random**: clear what you use. (The loader clears slot A; nothing else is cleared.)
12. **A DSR call changes R6-R9 and takes time**; keep the VRAM from `>3600` for the file service.
13. **The 9918 is not the F18A**: the F18A's extras (30 rows, per-cell colour, palette, enhanced colour, layers) are what

the kernel gives you; do not rely on a plain TI's video. `VMODE` **rewrites the screen**, and the attribute byte means roles in the normal colour modes and flags in the enhanced ones; **Graphics II and file I/O do not mix** (they share VRAM).

14. **The console's frame counter needs `>83FC`'s high byte non-zero** (the kernel sets it at start and after every DSR call). Do not clear it.
15. **Slot A is 7K.** If you need more, it is a SAMS page. **Music steps with the frames you let through**: a long loop that never calls `KIDLE` (or returns to the main loop) stalls the song.
16. **It works in the simulator** is the beginning of a test, not the end: js99er, then the real machine.

15. What the kernel does not do (version 1.5)

[TI] adds its own limits (the table above): one module of up to 4,096 bytes at `>C000`; no `data` pages, no music compiled at build (`MUPLAY` needs a song in pages; `SNDTONE` works), no databank of your own, no user macros or forward `EQU`, no cartridge image; variables in `APPRAM+5760` and above, so a program does not overwrite the workbench's own state (WORKBENCH.md part 5).

- Video: every F18A mode is reachable (`VMODE`, `VREG`), but the kernel only *draws* for you in the glyph modes (Text 80/40, Graphics I); sprites, Graphics II, Multicolor and the layers you write to VRAM yourself. No sprite library, no scroller, no tile editor. A module that changes registers beyond `VMODE`'s must keep `VR1CPY` right.
- Sound: a music player and tone effects (MUSIC.md), but no envelopes, no effect mixer and no priorities; the console's sound lists are available, unmixed with the player.
- No heap, no text-editing engine, no menus: those are applications (Press has an editor built on the ancestor of this kernel; see FROM-PRESS.md).
- At most three files open at once (the controller's three buffers); no relative-record files beyond what the PAB offers, no file-size query.
- Joysticks are read, but only as direction-and-fire switches (no analog, no presence test, no calibration).
- No cooperative multitasking: one module's `TICK` runs at a time.

Appendix: a second module in five minutes

1. `app.json`: add `{"name": "about", "source": "about.a99"}`.
2. `about.a99`: copy the scaffold; give it its own `INIT` that draws and `TICK` that, on any key, does `MOV @PREVPG,R0 / CLR R1 / BL @GOMOD`.
3. In `main.a99`'s `TICK`: on `CI R0,>8100` (CTRL A) do `LI R0,PG_ABOUT / CLR R1 / BL @GOMOD`.
4. Build and run. In the first module, make `INIT` redraw everything from slot A state, because it runs again when you come back.